

Universidad La Salle

Facultad de Ingenierías y Arquitectura

Plataforma de Pre-incubación Parmenia

Backend

Informe del Proyecto

Integrantes: Huamani Vásquez Juan José
Florez Gonzalez Fatima
Yabar Carazas Melvin Jarred
Zela Flores Gabriel Frank

Profesor: Machaca Arceda Vicente Enrique

22 de Mayo, 2026

Índice

1. Descripción Breve	3
2. Motivación	3
3. Requerimientos	3
3.1. Funcionales	3
3.2. No Funcionales	4
3.3. Tecnologías	4
4. Arquitectura	4
4.1. Visión General	4
4.2. Componentes	5
5. Esquema de Bases de Datos	6
5.1. Diagrama Entidad-Relación	6
5.1.1. profiles	7
5.1.2. cohorts	7
5.1.3. enrollments	7
5.1.4. phases	8
5.1.5. projects	8
5.1.6. project_members	8
5.1.7. project_mentors	9
5.1.8. posts	9
5.1.9. deliverables	9
5.1.10. deliverable_reviews	10

1. Descripción Breve

Este proyecto consiste en el desarrollo de un **backend MVP** para la plataforma de pre-incubación **Parmenia**. La plataforma digitaliza los procesos de gestión de la incubadora: administración de convocatorias, inscripción de emprendedores, gestión de proyectos y entregables, asignación de mentores y seguimiento del progreso.

La aplicación está construida con **FastAPI** (Python) como servidor web, **SQLAlchemy** como ORM, y **Supabase/PostgreSQL** como base de datos. La autenticación se gestiona con **JWT propios** generados por la API mediante la librería **python-jose**, con control de acceso basado en roles (administrador, mentor y emprendedor). Los usuarios se registran e inician sesión a través de los endpoints **/auth/register** y **/auth/login**, eliminando la dependencia de servicios de autenticación externos.

La API REST expone operaciones CRUD para todas las entidades del sistema, además de endpoints funcionales como la revisión de entregables con avance automático de fase y la gestión de miembros y mentores dentro de los proyectos.

2. Motivación

La motivación de este proyecto nace de la experiencia directa como parte de la incubadora **Parmenia**. Al ser una incubadora joven, con tan solo dos convocatorias realizadas y encontrándose actualmente en su segunda edición, fue evidente la necesidad de contar con un sistema que facilitara la gestión de la información y los procesos internos.

Actualmente, gran parte de las operaciones se realizan de forma manual: inscripciones, seguimiento de proyectos, asignación de mentores, revisión de entregables y comunicación con los participantes. Esta falta de digitalización genera ineficiencias y una experiencia confusa tanto para los organizadores como para los alumnos.

Las necesidades específicas identificadas fueron:

- **Falta de información centralizada:** No existe un lugar donde consultar qué es **Parmenia**, cómo funciona, cómo inscribirse y cuáles son los beneficios.
- **Proceso de inscripción poco claro:** Los alumnos no tienen un mecanismo sencillo para inscribirse a una convocatoria.
- **Gestión manual de proyectos:** Los organizadores administran proyectos, miembros y mentores de forma manual, dificultando el seguimiento.
- **Dificultad de coordinación:** La interacción entre administradores, mentores y emprendedores depende de herramientas externas como hojas de cálculo.

Este proyecto busca simplificar tanto la labor de los organizadores como la experiencia de los alumnos, proporcionando una plataforma accesible y funcional que acompañe el crecimiento de **Parmenia**.

3. Requerimientos

3.1. Funcionales

RF1 Gestión CRUD de perfiles de usuario con roles: administrador, mentor, emprendedor.

RF2 Gestión CRUD de convocatorias (nombre, descripción, fecha inicio, fecha fin).

RF3 Inscripción de emprendedores a convocatorias, con aceptación o rechazo por administrador.

- RF4** Gestión CRUD de fases del programa, ordenadas secuencialmente.
- RF5** Creación y gestión de proyectos, con asignación automática a la primera fase.
- RF6** Agregar/remover miembros de un proyecto y asignar/desasignar mentores.
- RF7** Subida, consulta, actualización y eliminación de entregables por proyecto y fase.
- RF8** Gestión CRUD de publicaciones informativas por administradores y mentores.
- RF9** Restricción de acceso a endpoints según el rol del usuario autenticado.
- RF10** Registro e inicio de sesión de usuarios mediante endpoints propios (`/auth/register`, `/auth/login`), con emisión y validación de tokens JWT.

3.2. No Funcionales

- RNF1** Toda comunicación sobre HTTPS. Endpoints protegidos por JWT y control de roles.
- RNF2** Arquitectura escalable que permita agregar funcionalidades sin reescritura significativa.
- RNF3** Disponibilidad mínima del 99.5 %, aprovechando la infraestructura de Supabase.
- RNF4** Código organizado por capas (routers, modelos, schemas, autenticación).
- RNF5** Documentación interactiva autogenerada (Swagger UI / OpenAPI).

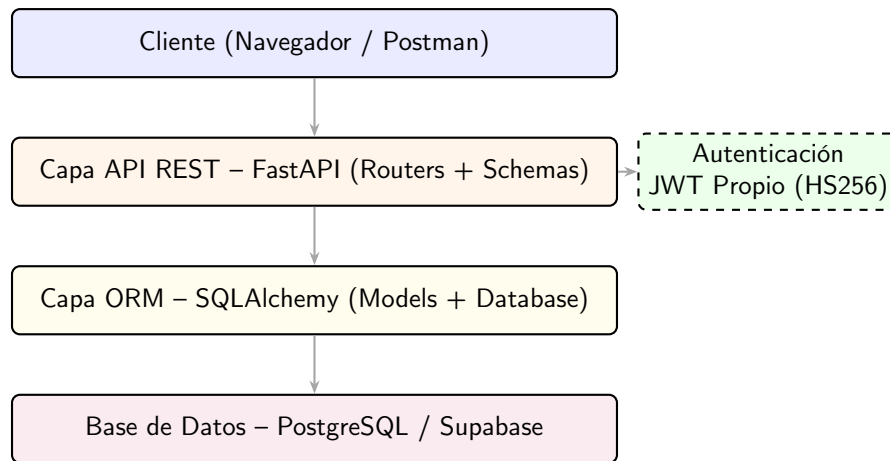
3.3. Tecnologías

1. Lenguaje: Python 3.10+
2. Framework: FastAPI
3. ORM: SQLAlchemy
4. Base de datos: PostgreSQL (Supabase)
5. Autenticación: JWT propio (python-jose, HS256)
6. Servidor ASGI: Uvicorn

4. Arquitectura

4.1. Visión General

La aplicación sigue una **arquitectura en capas** donde cada componente tiene una responsabilidad definida:



4.2. Componentes

Routers (Capa de Presentación): Definen los endpoints de la API REST. Cada entidad tiene su propio router:

- `auth.py` – Registro, inicio de sesión y perfil del usuario autenticado
- `profiles.py` – Perfiles de usuario
- `phases.py` – Fases del programa
- `cohorts.py` – Convocatorias
- `enrollments.py` – Inscripciones
- `projects.py` – Proyectos, miembros y mentores
- `deliverables.py` – Entregables y revisiones
- `posts.py` – Publicaciones informativas

Autenticación (`auth.py`): Gestiona el ciclo completo de autenticación JWT. El endpoint `/auth/register` crea un usuario con email y contraseña, y devuelve un token. El endpoint `/auth/login` valida credenciales y emite un nuevo token. La función `get_current_user()` decodifica el JWT en cada petición protegida, extrae el `sub` (UUID del usuario), busca el perfil en la base de datos y retorna un `AuthenticatedUser`. Las funciones auxiliares `require_roles()`, `is_project_member()`, `is_project_mentor()` y `can_access_project()` implementan la autorización basada en roles.

Modelos (`models.py`): Mapean las tablas de PostgreSQL a objetos Python mediante SQLAlchemy. Se definen los enums `UserRole`, `EnrollmentStatus` y `ReviewStatus`, y las relaciones entre entidades. El modelo `Profile` incluye campos de autenticación (`email`, `password`) y su `id` es un UUID autogenerado, sin dependencia de servicios externos.

Schemas (`schemas.py`): Definen la estructura de entrada/salida de cada endpoint con Pydantic: `*Create`, `*Update` y `*Response` por entidad. Incluye schemas de autenticación como `RegisterRequest`, `LoginRequest` y `AuthResponse`.

Base de datos (`database.py`): Configura la conexión mediante `DATABASE_URL` y proporciona la dependencia `get_db()` con sesión por petición.

5. Esquema de Bases de Datos

La base de datos consta de **10 tablas** en PostgreSQL (Supabase). Utiliza tipos nativos como **UUID**, **ARRAY** y tipos enumerados personalizados (**user_role**, **enrollment_status**, **review_status**). La tabla **profiles** es autónoma y gestiona las credenciales de usuario directamente, sin dependencia de **auth.users** de Supabase.

5.1. Diagrama Entidad-Relación

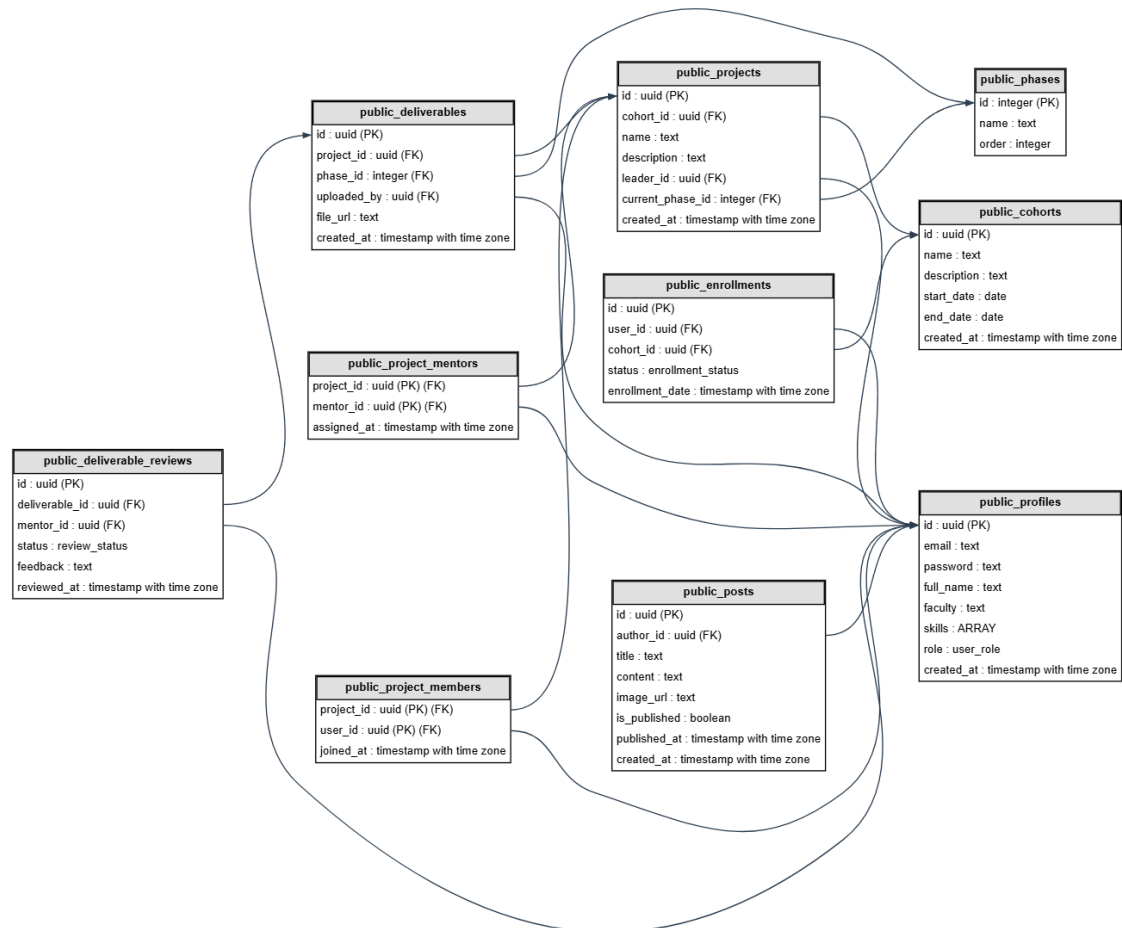


Figura 1: Diagrama Entidad Relacion del Proyecto

5.1.1. profiles

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico del usuario, autogenerado por la base de datos
email	TEXT (UNIQUE)	Correo electrónico del usuario, utilizado para inicio de sesión
password	TEXT	Contraseña del usuario en texto plano (MVP)
full_name	TEXT	Nombre completo del usuario
faculty	TEXT	Facultad a la que pertenece el usuario
skills	ARRAY[TEXT]	Lista de habilidades del usuario, ej: ["Python", "Diseño UX"]
role	ENUM(user_role)	Rol del usuario: admin, mentor o emprendedor. Por defecto: emprendedor
created_at	TIMESTAMPTZ	Fecha de creación del registro (auto)

Tabla 1: Estructura de datos: profiles

5.1.2. cohorts

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico de la convocatoria
name	TEXT	Nombre de la convocatoria, ej: “Convocatoria 2026-I”
description	TEXT	Descripción de la convocatoria y sus objetivos
start_date	DATE	Fecha de inicio de la convocatoria
end_date	DATE	Fecha de finalización de la convocatoria
created_at	TIMESTAMPTZ	Fecha de creación del registro (auto)

Tabla 2: Estructura de datos: cohorts

5.1.3. enrollments

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico de la inscripción
user_id	UUID (FK → profiles)	Usuario que se inscribe a la convocatoria
cohort_id	UUID (FK → cohorts)	Convocatoria a la que se inscribe el usuario
status	ENUM(enrollment_status)	Estado de la inscripción: pendiente, aceptada o rechazada. Por defecto: pendiente
enrollment_date	TIMESTAMPTZ	Fecha en que se realizó la inscripción (auto)

Tabla 3: Estructura de datos: enrollments

5.1.4. phases

Campo	Tipo	Descripcion
id	INTEGER (PK, IDENTITY)	Identificador autoincremental de la fase
name	TEXT	Nombre de la fase, ej: “Ideación”, “Validación”, “Desarrollo”
order	INTEGER (UNIQUE)	Posición ordinal de la fase en la secuencia del programa

Tabla 4: Estructura de datos: phases

5.1.5. projects

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico del proyecto
cohort_id	UUID (FK → cohorts)	Cohorte a la que pertenece el proyecto
name	TEXT	Nombre del proyecto
description	TEXT	Descripción del proyecto emprendedor
leader_id	UUID (FK → profiles)	Emprendedor líder que creó el proyecto
current_phase_id	INTEGER (FK → phases)	Fase actual del proyecto. Por defecto: 1
created_at	TIMESTAMPTZ	Fecha de creación del registro (auto)

Tabla 5: Estructura de datos: projects

5.1.6. project_members

Campo	Tipo	Descripcion
project_id	UUID (PK, FK → projects)	Proyecto al que pertenece el miembro
user_id	UUID (PK, FK → profiles)	Perfil del usuario miembro del proyecto
joined_at	TIMESTAMPTZ	Fecha de incorporación del miembro al proyecto (auto)

Tabla 6: Estructura de datos: project_members

5.1.7. project_mentors

Campo	Tipo	Descripcion
project_id	UUID (PK, FK → projects)	Proyecto al que está asignado el mentor
mentor_id	UUID (PK, FK → profiles)	Perfil del mentor asignado al proyecto
assigned_at	TIMESTAMPTZ	Fecha de asignación del mentor al proyecto (auto)

Tabla 7: Estructura de datos: project_mentors

5.1.8. posts

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico de la publicación
author_id	UUID (FK → profiles)	Autor de la publicación
title	TEXT	Título de la publicación
content	TEXT	Contenido de la publicación
image_url	TEXT	URL de imagen adjunta (opcional)
is_published	BOOLEAN	Indica si está publicada o en borrador. Por defecto: true
published_at	TIMESTAMPTZ	Fecha de publicación efectiva
created_at	TIMESTAMPTZ	Fecha de creación del registro (auto)

Tabla 8: Estructura de datos: posts

5.1.9. deliverables

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico del entregable
project_id	UUID (FK → projects)	Proyecto al que pertenece el entregable
phase_id	INTEGER (FK → phases)	Fase del programa en la que se subió el entregable
uploaded_by	UUID (FK → profiles)	Usuario que subió el entregable
file_url	TEXT	URL del archivo entregado (PDF, presentación, enlace, etc.)
created_at	TIMESTAMPTZ	Fecha de creación del registro (auto)

Tabla 9: Estructura de datos: deliverables

5.1.10. deliverable_reviews

Campo	Tipo	Descripcion
id	UUID (PK)	Identificador unico de la revisión
deliverable_id	UUID (FK → deliverables)	Entregable que se está revisando
mentor_id	UUID (FK → profiles)	Mentor que realiza la revisión
status	ENUM(review_status)	Estado de la revisión: pendiente, aprobado o rechazado. Por defecto: pendiente
feedback	TEXT	Retroalimentación del mentor sobre el entregable
reviewed_at	TIMESTAMPTZ	Fecha en que se realizó la revisión (auto)

Tabla 10: Estructura de datos: deliverable_reviews